

C/C++ Compiler

Es gibt eine Vielzahl an Compilern für C und C++. Auch wenn sie sich in Details unterscheiden, verfolgen sie alle dasselbe Ziel: menschlich lesbaren Quellcode in ausführbaren Maschinencode zu verwandeln.

An dieser Stelle stolpern viele Einsteiger über die erste Hürde: Im Gegensatz zu Sprachen wie Java oder Python ist C++ nicht von Haus aus plattformunabhängig. Der erzeugte Maschinencode ist strikt an die jeweilige Hardware-Architektur (z. B. x64 oder ARMv8) und das Betriebssystem gebunden.

Viele moderne Programmiersprachen nehmen dem Entwickler die Arbeit unter der Haube komplett ab. Wer von Java kommt, ist es gewohnt, auf "Ausführen" zu drücken – den Rest erledigt die Runtime oder die Virtual Machine. C und C++ hingegen gelten als systemnahe Sprachen. Sie geben dir die volle Kontrolle, verlangen dafür aber auch ein tieferes Verständnis der Prozesse. Wer nicht versteht, was zwischen dem Schreiben des Codes und dem fertigen Programm passiert, wird spätestens bei Linker-Fehlern oder plattformspezifischen Bugs verzweifeln.

Damit du nicht nur blind auf den Button klickst, schauen wir uns jetzt an, was im Hintergrund wirklich passiert – vom Präprozessor bis hin zum fertigen Binary.

Die Drei Phasen zur ausführbaren Datei.

Um zu verstehen, wie aus deinem Text eine startbare Anwendung wird, zerlegen wir den Prozess mit MinGW64 im Terminal in seine Einzelteile. Unser Testobjekt ist die main.cpp-Datei

```
#include <iostream>
int main()
{
    std::cout << "hello World" << std::endl;
    return 0;
}
```

1. Kompilierung (C++ zu Assembly)

In diesem Schritt übersetzt der eigentliche Compiler den C++ Code in Assembler Sprache. Sie ist für Menschen sehr schwer lesbar und sehr hardwarenah.

- Befehl im Terminal: `g++ -S main.cpp -o main.s`
- Hier entsteht eine Assembly Datei main.s, wie das funktioniert wird später genauer erleutert.

2. Assembler (Assembly zu Maschinensprache)

Jetzt wird es für den Menschen unlesbar, der Assembler wandelt die .s-Datei in eine Objekt-Datei um. Das ist bereits Maschinencode (Binärcode) allerdings noch nicht ausführbar.

- Befehl im Terminal: `as -o main.o main.s`
- In der main.o Datei stehen nun die Anweisungen an den Prozessor, diese kann ein text-Editor meist gar nicht vernünftig darstellen.
- Hier fehlen noch die Verknüpfungen zu Bibliotheken, vor allem zur Standard-Library

3. Das Linken (Objekt-Datei zu Executable)

Der Linker vereint die vielen einzelnen Dateien und Bibliotheken aus denen ein Programm besteht und erzeugt eine .exe Datei (unter Windows)

- Befehl im Terminal: `g++ -o main main.o`

Nun ist eine main.exe Datei entstanden, diese kann im Terminal ausgeführt werden mit: `./main`

Will man das Programm mit einem mal kompilieren und ausführen, sollte man: `g++ main.cpp -o main` verwenden

Wichtig ist es, beim kompilieren dem Compiler alle Dateien, die man verwenden möchte auch übergeben.

Beispiel:

main.cpp:

```
#include <iostream>
#include "helper.h"
int main()
{
    std::cout << a << std::endl;
    return 0;
}
```

helper.h:

```
#ifndef HELPER_H
#define HELPER_H
    extern int a;
#endif
```

helper.cpp:

```
#include "helper.h"
int a = 10;
```

hat man also zwei .cpp Dateien sieht der Befehl so aus: `g++ main.cpp helper.cpp -o main`

hier wird jede cpp Datei einzeln kompiliert und der Linker führt Beide am Ende zusammen, header-Dateien werden bereits von Präprozessor in den Quellcode Kopiert, bevor die eigentliche Kompilierung beginnt.

Funktionsweise des eigentlichen Compilers

Der Compiler muss den C++ Quellcode in Assembly umwandeln, dieser Prozess läuft in einer vorgegebenen Struktur ab:

1. Präprozessor

Der Präprozessor löst alle Direktiven auf, die mit # beginnen. Z.B. `#include <file>` wird durch den Inhalt dieser Datei ersetzt

- Präprozessordirektiven beginnen mit #
- Beispiele: `#include`, `#define`, `#ifndef`, `#ifdef`

2. Lexikalische Analyse (Scanning)

Quelltext wird in kleine sinnvolle Einheiten (Tokens) zerlegt

- Z.B.: `int x = 5;` wird zu `[KEYWORD: int]`, `[IDENTIFIER: x]`, `[OPERATOR: =]`, `[LITERAL: 5]`, `[PUNCTUATOR: ;]`

3. Syntaktische Analyse (Parsing)

Überprüfung ob die Tokens die Grammatik von C++ einhalten

- Token werden in eine hierarchische Struktur gebracht (Abstract Syntax Tree -> AST)

4. Semantische Analyse

- Typprüfung
- Symboltabelle
- Variablenbindung

5. Optimierung und Codeerzeugung

- Compiler erarbeitet eine Art Zwischensprache zur Optimierung (Intermediate Representation – IR)
- Das optimierte IR wird in den Maschinencode übersetzt

Als kurzes, vereinfachtes Beispiel zur Erklärung der Ausdruck: $-a * 3 + b / (4 - c)$ soll verarbeitet werden hierfür wird die EBNF-Grammatik der arithmetischen Operatoren verwendet.

Expr = ["+" | "-"] Term {"+" | "-"} Term}.

Term = Factor {"*" | "/" } Factor}.

Factor = ident | number | "(" Expr ")".

Regeln für Metasymbole:

| trennt alternativen

(...) gruppiert alternativen

[...] Option

{...} Wiederholung 0 bis unendlich

Terminalsymbole sind in Anführungszeichen ("...")

Lexikalische Analyse (Scanner)

Token Typ

-	MINUS (Vorzeichen/Operator)
a	IDENTIFIER (Variable)
*	MULTIPLY (Operator)
3	NUMBER (Literal)
+	PLUS (Operator)
b	IDENTIFIER (Variable)
/	DIVIDE (Operator)
(	L_PAREN (Klammer auf)
4	NUMBER (Literal)
-	MINUS (Operator)
c	IDENTIFIER (Variable)
)	R_PAREN (Klammer zu)

Syntaktische Analyse (Parser)

- ident	*	number	+	ident	/	(Expr)
- Factor	*	Factor	+	Factor	/	Factor
-		Term	+			Term
						Expr

AST:

```
[+] (Root: Expr)
 / \
[*]  [/] (Terme)
 /\  /\
[-] 3 b [-] (Klammer-Expr wird zum Teilbaum)
 |      /\
a      4 c
```

Intermediate Representation

Ausdruck: $-a * 3 + b / (4 - c)$ vereinfachte IR:

```
t1 = neg a
t2 = t1 * 3
t3 = 4 - c
t4 = b / t3
result = t2 + t4
```

Codeerzeugung für x64 Windows system:

```
mov eax, [a]    ; Lade a in Register
neg eax         ; t1 = -a
imul eax, 3     ; t2 = t1 * 3
... und so weiter ...
```

Nun muss der Assembler starten. Er verwendet im Wesentlichen einfach eine Lookup-Tabelle und ersetzt die Befehle durch deren äquivalenten Operation-Code:

Bsp.: mov

1. Scanner erkennt Token mov
2. Operand Check: Quelle und Ziel sind ein 32-Bit-Register (eax, ebx)
3. Table Lookup: MOV das von einem 32-Bit Register in ein anderes 32-Bit Register ist, Basis-Opcode: 0x89
4. Encoding: Assembler muss ein zusätzliches Byte berechnen (ModR/M-Byte) um dem Prozessor mitzuteilen, dass EAX und EBX verwendet werden sollen
5. In die Objekt-Datei wird geschrieben 89 D8
6. Assembler erstellt eine Symbol Tabelle (Speichert Labels und deren relative Adressen) und eine Relocation Table (zur Speicherung unbekannter Adressen)

Der Assembler schreibt beim Aufruf externer Funktionen einen Platzhalter, den der Linker auffüllt, da dieser festlegt, wo sich später die Speicheradresse der Funktion ist

Der Linker nimmt nun die Objekt-Dateien und erstellt hieraus die executable.

1. Sektionen zusammenführen
 - Code-Abschnitte werden hintereinandergelegt, dadurch verschieben sich interne Adressen
2. Adressen patchen
 - Linker findet die unbekannten Adressen heraus, die in der Relocation Table fehlen und füllt sie.

Der Linker sorgt dafür, dass die Objekt-dateien ausführbar sind und erstellt eine executable Datei.

Compiler	Entwickler	Haupt-Einsatzbereich	Besonderheiten
GCC (g++)	GNU Project	Linux, Embedded Systems, Open Source	Der "Goldstandard" in der Open-Source-Welt. Sehr stabil und unterstützt fast jede Hardware-Architektur.
Clang / LLVM	Apple, Google, u.a.	macOS, iOS, Android, Tool-Entwicklung	Bekannt für extrem hilfreiche Fehlermeldungen und eine modulare Architektur. Basiert auf der LLVM-Infrastruktur.
MSVC	Microsoft	Windows (Desktop-Apps, Spiele)	Der Standard-Compiler für die Windows-Entwicklung, tief integriert in Visual Studio.
MinGW-w64	Community	Windows (Portierung von GCC)	Eine Portierung des GCC für Windows. Ideal, wenn man Linux-ähnliche Befehle (wie in deinem Artikel) unter Windows nutzen will.
Intel C++ Compiler (ICX)	Intel	High-Performance Computing (HPC)	Speziell darauf optimiert, das Maximum aus Intel-CPU's herauszuholen (z.B. für wissenschaftliche Berechnungen).